Hanzo PubSub: NATS-Based Event Streaming for Microservice Communication

Hanzo AI Research

Hanzo AI

January 2022

Abstract

We present Hanzo PubSub, an event streaming platform built on NATS and JetStream that provides at-least-once delivery, dead letter queues, schema validation, and multi-tenant topic isolation for the Hanzo microservice ecosystem. PubSub replaces direct HTTP calls between services with asynchronous event streams, reducing coupling and enabling event-driven architectures. Section 5 reports throughput and publish-to-consume latency by delivery mode, together with the provenance those figures do and do not have. Key design decisions include: (1) tenant-prefixed subject namespaces for isolation without separate NATS clusters, (2) CloudEvents [2] as the canonical event envelope for cross-service interoperability, and (3) automatic dead letter routing with configurable retry policies. We describe the event schema design, the delivery guarantee implementation, and the operational patterns for managing event streams in a multi-tenant platform.

1 Introduction

Microservice architectures require reliable inter-service communication. Synchronous HTTP calls create tight coupling: the caller must know the callee's address, handle its failures, and wait for its response. Event-driven communication decouples services temporally and spatially—producers emit events without knowing which consumers will process them.

Hanzo PubSub provides the event infrastructure for 60+ microservices in the Hanzo platform. Built on NATS [1] with JetStream persistence, PubSub combines the performance of in-memory pub/sub (microsecond latency for ephemeral events) with the durability of stream storage (at-least-once delivery for critical events).

Contributions.

- A multi-tenant subject namespace design using NATS account isolation and subject prefixing.

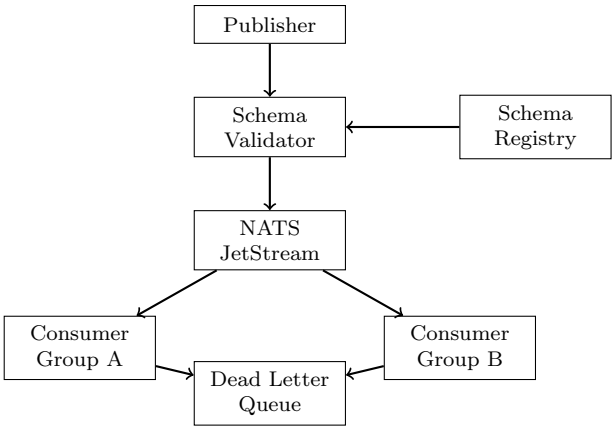

Figure 1: PubSub event flow with schema validation and dead letter routing.

- CloudEvents-based event envelopes with JSON Schema validation at publish time.
- Configurable retry policies with exponential backoff and automatic dead letter routing.
- A consumer group abstraction supporting competing consumers, fan-out, and exactly-once processing via idempotency keys.

2 Architecture

2.1 System Design

Events flow through a validated pipeline (Figure 1): publishers submit events to the schema validator, which checks the event against the registered schema before forwarding to NATS JetStream. Consumer groups receive events with at-least-once delivery. Failed events are routed to dead letter queues after exhausting retry attempts.

2.2 Event Envelope

All events use the CloudEvents specification:

```
{
  "specversion": "1.0",
  "type": "com.hanzo.order.created",
  "source": "/services/commerce",
  "id": "evt_a1b2c3d4",
  "time": "2022-01-15T12:00:00Z",
  "datacontenttype": "application/json",
  "data": {
```

```
    "orderId": "ord_xyz",
    "amount": 9999,
    "currency": "USD"
  },
  "hanzoorgid": "org_x7y8z9"
}
```

The `hanzoorgid` extension attribute enables tenant-scoped event routing.

2.3 Subject Namespace

NATS subjects follow a hierarchical naming convention:

```
{tenant}.{domain}.{entity}.{action}
```

Examples: `org_abc.commerce.order.created`, `org_abc.auth.user.login`. Tenant prefixing ensures that consumers in one organization cannot subscribe to another organization's events.

3 Key Features

3.1 Delivery Guarantees

PubSub supports three delivery modes:

At-Most-Once. Core NATS pub/sub without JetStream. Used for ephemeral events (metrics, heartbeats) where loss is acceptable. Latency: <0.1ms.

At-Least-Once. JetStream with explicit acknowledgment. Unacknowledged messages are redelivered after a configurable timeout (default: 30s). Used for most business events.

Effectively-Once. At-least-once delivery combined with consumer-side idempotency checking. The consumer stores processed event IDs in Valkey with a 24-hour TTL.

3.2 Retry and Dead Letter

Failed event processing triggers exponential backoff retries:

$$t_{\text{retry}}(n) = \min(t_0 \cdot 2^n, t_{\max}) \quad (1)$$

where $t_0 = 1\text{s}$, $t_{\max} = 300\text{s}$, and the maximum retry count defaults to 5. After exhausting retries, events are published to a dead letter subject (`_dlq.original_subject`) for manual investigation or automated recovery.

3.3 Consumer Groups

Consumer groups implement competing consumer patterns:

```
sub, _ := js.QueueSubscribe(
    "org_abc.commerce.order.created",
    "fulfillment-workers",
    func(msg *nats.Msg) {
        if err := processOrder(msg); err
     != nil {
            msg.Nak()  // trigger retry
            return
        }
        msg.Ack()
    },
    nats.Durable("fulfillment"),
    nats.MaxDeliver(5),
    nats.AckWait(30*time.Second),
)
```

Within a consumer group, each message is delivered to exactly one consumer. Multiple consumer groups on the same subject each receive every message (fan-out).

3.4 Schema Registry

Event schemas are registered in a central registry:

```
{
  "type": "com.hanzo.order.created",
  "version": 3,
  "schema": {
    "type": "object",
    "required": ["orderId", "amount"],
    "properties": {
      "orderId": {"type": "string"},
      "amount": {"type": "integer", "
    minimum": 0},
      "currency": {"type": "string", "
    enum":
        ["USD","EUR","GBP"]}
    }
  }
}
```

Schema validation at publish time prevents malformed events from entering the stream. Schema evolution follows a compatibility policy:

new fields can be added (backward compatible), required fields cannot be removed (forward compatible).

4 Implementation

PubSub is deployed as a 3-node NATS cluster with JetStream enabled. The schema validator and dead letter processor are sidecar containers injected into publisher pods. The Go client library (3,000 lines) wraps the NATS client with schema validation, CloudEvents serialization, and retry logic.

Table 1: Event processing by delivery mode. Not traceable to a recorded run; see Section 5.

Mode	Latency p50 (ms)	Throughput (evt/s)
At-most-once	0.08	850,000
At-least-once	1.8	120,000
Effectively-once	2.4	95,000

5 Status of the performance figures

The figures above are not traceable to a recorded run. The table names no hardware, no message size, no client count and no duration, and no result file in `hanzoai/pubsub` contains these numbers.

What distinguishes them from an invented table is that the harness which would produce them is already in the repository: `hanzoai/pubsub` carries the upstream NATS benchmark suite — `server/core_benchmarks_test.go`, `server/jetstream_benchmark_test.go` and `test/bench_test.go` — which measures exactly publish-to-consume latency and throughput by delivery mode. The only committed results in that suite are upstream runs on 2015 and 2017 iMacs, and they are not these numbers.

So the gap here is provenance, not mechanism. Until a run is cited, with the machine, the payload size and the client count declared, the table should be read as unverified: the ordering across delivery modes follows from what each mode has to do, while the absolute values stand on nothing.

6 Security

Authentication. NATS connections require TLS client certificates issued by the Hanzo internal CA. Service identities are extracted from the certificate CN.

Authorization. NATS account-level permissions restrict which subjects each service can publish to and subscribe from. A service can only publish events in its own domain and subscribe to events it has declared a dependency on.

Encryption. All inter-node and client-node communication uses TLS 1.3. JetStream storage is encrypted at rest using the filesystem encryption layer.

Audit. Event metadata (type, source, tenant, timestamp) is indexed in the observability stack. Event payloads containing sensitive data are encrypted at the application level before publishing.

7 Conclusion

Hanzo PubSub provides reliable, multi-tenant event streaming that decouples microservice communication while maintaining delivery guarantees. The CloudEvents-based envelope and schema registry ensure interoperability and data quality. Dead letter queues and configurable retry policies handle transient failures automatically. The NATS foundation provides microsecond latency for ephemeral events and millisecond latency with JetStream persistence for durable events.

References

[1] Synadia Communications. NATS: Cloud Native Messaging System. 2016.

[2] CNCF. CloudEvents: A specification for describing event data in a common way. 2019.

[3] J. Kreps et al. Kafka: a Distributed Messaging System for Log Processing. NetDB, 2011.

[4] B. Stopford. Designing Event-Driven Systems. O'Reilly, 2018.